

CenterCLIP: Token Clustering for Efficient Text-Video Retrieval

Shuai Zhao
CCAI, Zhejiang University
Hangzhou, China
zhaoshuaimcc@gmail.com

Xiaohan Wang
CCAI, Zhejiang University
Hangzhou, China
wxh1996111@gmail.com

Linchao Zhu
ReLER Lab, AAIL, University of Technology Sydney
Sydney, Australia
zhulinchao7@gmail.com

Yi Yang
CCAI, Zhejiang University
Hangzhou, China
yangyics@zju.edu.cn

ABSTRACT

Recently, large-scale pre-training methods like CLIP have made great progress in multi-modal research such as text-video retrieval. In CLIP, transformers are vital for modeling complex multi-modal relations. However, in the vision transformer of CLIP, the essential visual tokenization process, which produces discrete visual token sequences, generates many homogeneous tokens due to the redundancy nature of consecutive and similar frames in videos. This significantly increases computation costs and hinders the deployment of video retrieval models in web applications. In this paper, to reduce the number of redundant video tokens, we design a **multi-segment token clustering algorithm** to find the **most representative tokens** and **drop the non-essential** ones. As the frame redundancy occurs mostly in consecutive frames, we divide **videos into multiple segments** and **conduct segment-level clustering**. Center tokens from each segment are **later concatenated into a new sequence**, while their original **spatial-temporal relations** are well maintained. We instantiate **two clustering algorithms** to efficiently find deterministic medoids and iteratively partition groups in high dimensional space. Through this token clustering and center selection procedure, we successfully reduce computation costs by removing redundant visual tokens. This method further enhances segment-level semantic alignment between video and text representations, enforcing the spatio-temporal interactions of tokens from within-segment frames. Our method, coined as CenterCLIP, surpasses existing state-of-the-art by a large margin on typical text-video benchmarks, while reducing the training memory cost by 35% and accelerating the inference speed by 14% at the best case. The code is available at <https://github.com/mzhaoshuai/CenterCLIP>.

CCS CONCEPTS

• **Information systems** → *Novelty in information retrieval.*

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SIGIR '22, July 11–15, 2022, Madrid, Spain

© 2022 Association for Computing Machinery.

ACM ISBN 978-1-4503-8732-3/22/07...\$15.00

<https://doi.org/10.1145/3477495.3531950>



Figure 1: t-SNE [50] visualization of video token embeddings of CLIP. The shown similar image patches within a cluster are from different temporal frames in the same video. Best viewed in color with 300% zoom.

KEYWORDS

Text-video retrieval; CLIP; transformer; token clustering

ACM Reference Format:

Shuai Zhao, Linchao Zhu, Xiaohan Wang, and Yi Yang. 2022. CenterCLIP: Token Clustering for Efficient Text-Video Retrieval. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '22)*, July 11–15, 2022, Madrid, Spain. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3477495.3531950>

1 INTRODUCTION

Text-video retrieval is less studied than the commonly known text-image retrieval task as the intricate context of the video, especially when the length of the video is very long or the temporal variation of the video is large. With the explosive growth of video content on mobile phones and Internet during the past decade, text-video retrieval becomes increasingly popular. People also desire a better text-video retrieval system as searching for videos of interest already becomes a part of daily lives of most people.

Recently, with the success of large-scale contrastive language-image pre-training methods like CLIP [39], text-video retrieval also has made great progress. To be specific, CLIP4clip [32] transfers the knowledge of CLIP to text-video retrieval tasks, surpassing the previous state-of-the-art methods by a large margin (*e.g.*, more than 30% improvement of the recall metric on ActivityNet [12]). This demonstrates the power of billion-scale image-text pairs pre-training via contrastive learning. In CLIP, a vision transformer [11, 51] is adopted for visual representation learning. Typically, in vision transformer, visual tokenization, *i.e.*, linear projection of non-overlapped image patches to an embedding space, is a necessary component to produce discrete visual token sequences. Then token sequences can be processed by the multi-head self-attention (MHSA) in transformer blocks as the same manner of dealing with text sequences in the original transformer [51].

When the input of the vision transformer becomes videos, the visual tokenization procedure produces many homogeneous tokens due to the redundancy nature in continuously changing frames. In Figure 1, we extract the token embedding of CLIP from different frames in the same video and visualize them by t-SNE [50]. From the visualization, we can see those token embeddings from different frames form many tight clusters. Image patches with similar texture features correspond to immediate data points within a certain cluster. It is also clear that the number of clusters and the average number of tokens in clusters are not small, *i.e.*, there are many similar token embedding in high-dimensional space. As a result, repeated computation of these homogeneous tokens in CLIP inevitably introduces a lot of unnecessary computation costs and hinders the training and deployment of video retrieval models in web applications. To resolve the above problem, in this work, we propose to distinguish the most representative tokens, *i.e.*, the center token of each cluster in Figure 1, and only use these typical tokens for visual representation learning as these tokens contribute most to the discriminative feature representation learning.

We introduce a multi-segment token clustering algorithm to find the most representative tokens to reduce computation costs, and achieve segment-level semantic alignment of video and text representation. An input video is divided into multiple temporal segments. Each segment contains the same number of consecutive frames. Given the token embeddings of these frames, a clustering algorithm is performed on each segment independently. After clustering, only center tokens of clusters are reserved and non-center tokens are dropped to avoid duplicated computation of similar tokens. This significantly reduces computation costs. Center tokens from the same temporal segment are then concatenated into a new visual sequence and arranged according to their original spatial-temporal positions, *i.e.*, tokens whose image patches occur earlier in the video would appear at the earlier position of the new visual sequence. Then the new visual sequence is processed by the standard transformer blocks. This enables the model to learn segment-level video representation via attention among tokens from within-segment frames. These segment-level video representations are aligned with the text through contrastive learning.

In this work, we introduce two instances of clustering algorithm in multi-segment token clustering. One is k-medoids equipped with a deterministic centroids initialization method, *i.e.*, KKZ initialization [23, 48], to ensure the clustering results are consistent through

multiple runs. A good initialization also helps the clustering algorithm converge fast. The other is spectral clustering which suits high-dimensional data points clustering. With our multi-segment token clustering algorithm, CenterCLIP achieves state-of-the-art performance on four common benchmarks: MSR-VTT [60], MSVD [7], LSMDC [42], and ActivityNet [12]. We achieve significant improvement of retrieval metrics on all these four datasets compared to the baseline. At the same time, we achieve a decent reduction in memory cost and speed up the inference process. Specifically, on ActivityNet, we achieve a 35% reduction in memory cost and 14% speedup of inference speed compared to the baseline.

2 RELATED WORKS

Contrastive Vision-Language Pre-Training. Since the success of derivative works of Contrastive Language-Image Pre-Training (CLIP) [39] in different areas [4, 32, 37, 47, 59], visual representation learning under text supervision attracts widespread attention. Huge models pre-trained on billion-scale image-text pairs from web like WenLan [18], Google’s ALIGN [21], and Microsoft’s Florence [65] emerged. In the language-video understanding area, there are similar works like Frozen in Time [2] and HowTo100M [35]. However, the scale of language-video pre-training is much smaller than language-image pre-training as the former is much more expensive. Following CLIP4clip [32], we transfer the knowledge of CLIP to the text-video retrieval task in this work.

Text-video Retrieval. Text-video retrieval is more complex than commonly studied text-image retrieval as the additional temporal dimension introduces complex context information. Previously, standard language-video learning methods tend to design dedicated fusion manners for cross-modal learning from offline extracted video and text features [20, 24, 26, 58, 62, 64]. Recently, the paradigm of end-to-end large-scale pre-training plus task-specific finetune becomes more and more popular for language-video understanding, *e.g.*, HowTo100M [35], MIL-NCE [34], ActBERT [69], VideoBERT [49], MMT [13], and HERO [28]. These methods achieve promising results on many language-video tasks and demonstrate the effectiveness of pre-training. Our work is also in this line, the difference is that we inherit the knowledge from CLIP [39], which is pre-trained on image-text pairs rather than video-text pairs.

Efficient Transformer. Recently, transformer becomes the unified model for many vision and text tasks [11, 19, 30, 40, 51]. However, there are many time-consuming operations in transformers such as self-attention and softmax operations. Some works try to reduce the complexity of self-attention for very long sequences or remove the softmax operation, *e.g.*, Performer [9], Linear Transformer [22], Linformer [55], Reformer [25], Sparse Transformer [8], Routing Transformer [44], Longformer [3], and Galerkin Transformer [6]. Very recently, in computer vision, people also notice that not all tokens matter for the final performance of the model. To reduce the computation cost, researchers try to learn a few most representative visual tokens [45, 57], learn to rank all tokens and select the most important ones [53], and learn to mask the unimportant tokens [41, 61]. Compared to these mentioned methods, we are parameter-free and introduce segment level semantic alignment of text and video representation for text-video retrieval.

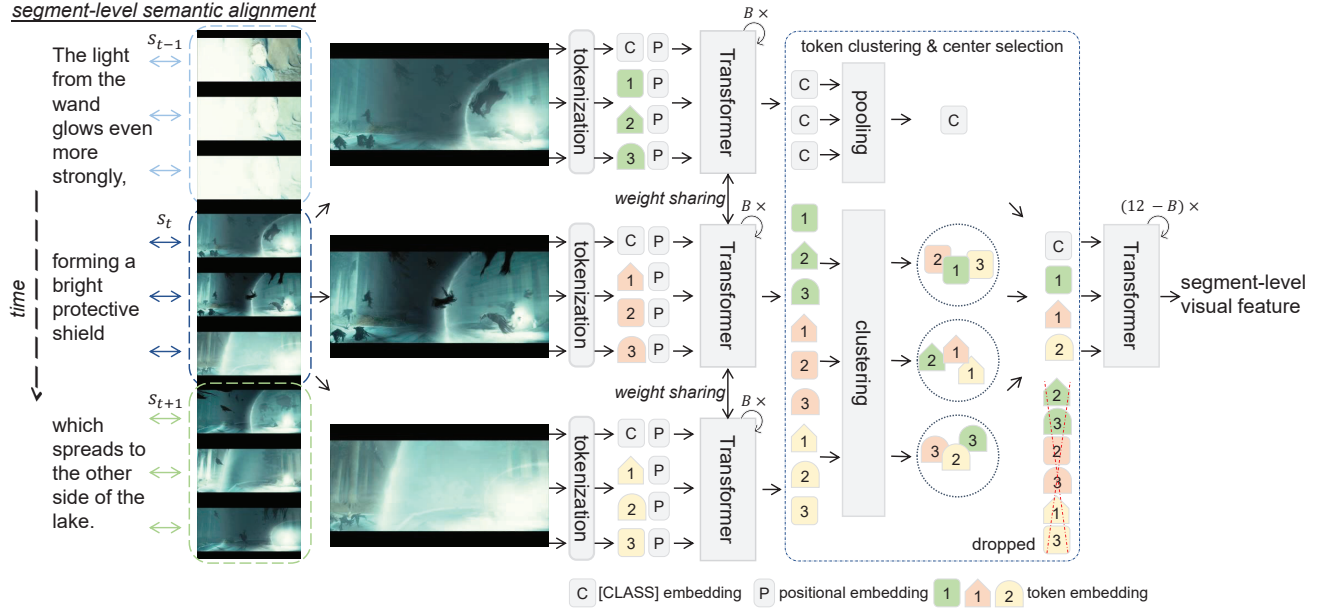


Figure 2: The overall framework of CenterCLIP and multi-segment clustering strategies. In this case, the video is divided into three segments and each contains three frames. Clustering is performed independently on tokens of each segment, and center tokens of all clusters from one segment are selected and concatenated into a new sequence. Via attention on this new sequence, the visual model is able to learn features that contain segment-level video semantics. This helps the whole text-video retrieval model to achieve segment-level semantic alignment between video and text while reducing computation costs.

3 METHODS

3.1 Preliminary

Given a video set $\mathcal{V}$ and text set $\mathcal{T}$, the goal of text-video retrieval is to learn a score function f , which gives a high similarity score $f(v_i, t_i)$ if a video $v_i \in \mathcal{V}$ and a text $t_i \in \mathcal{T}$ are highly relevant and a low similarity score for an irrelevant video-text pair. Then we can rank videos according to the query text (text to video retrieval) or rank texts according to the query video (video to text retrieval).

Following the typical multi-modal retrieval frameworks [27, 39], our text-video retrieval model is composed of a text encoder g and a video encoder h . Given a text t_i and a video v_i , $g(t_i)$ and $h(v_i)$ produce the *normalized* high-dimensional feature of the input, where ℓ_2 normalization is often considered in final feature encoding. Then the similarity score of this text-video pair (v_i, t_i) is

$$f(v_i, t_i) = h(v_i)^T g(t_i). \quad (1)$$

In training, a video-text pair (v_i, t_i) is treated as the positive if v_i and t_i are corresponded. All other instances of video or text in the mini-batch are treated as the negative. The text and video encoders are optimized in an end-to-end manner via normalized softmax loss [66]. The overall loss $\mathcal{L}$ is the average of video-to-text classification loss ($\mathcal{L}_{v2t}$) and text-to-video classification loss ($\mathcal{L}_{t2v}$):

$$\mathcal{L}_{v2t} = -\frac{1}{N} \sum_i \log \frac{\exp(h(v_i)^T g(t_i)/\tau)}{\sum_{j=1}^N \exp(h(v_i)^T g(t_j)/\tau)}, \quad (2)$$

$$\mathcal{L}_{t2v} = -\frac{1}{N} \sum_i \log \frac{\exp(g(t_i)^T h(v_i)/\tau)}{\sum_{j=1}^N \exp(g(t_i)^T h(v_j)/\tau)}, \quad (3)$$

$$\mathcal{L} = \frac{1}{2} (\mathcal{L}_{v2t} + \mathcal{L}_{t2v}), \quad (4)$$

where N is the mini-batch size and τ is the temperature to scale the logits. It is worth noting that τ is crucial because both $h(v_i)$ and $g(t_i)$ are normalized. We set it as a trainable parameter following the CLIP model. During training, our model is initialized from the pre-trained weight of CLIP. We describe the details of the text encoder and video encoder below.

3.1.1 Text encoder. We instantiate the text encoder using the text model of CLIP. It is a transformer [51] with the architecture modifications described in BERT [40], *i.e.*, only encoder and no decoder. A transformer model typically consists of repeated blocks (layers) of multi-head self-attention (MHSA) and feed-forward networks (FFN). We use a transformer with 12 layers and 512 width with 8 attention heads, where the width is the dimension of the query, key, and value feature. The text tokenizer is a lower-cased byte pair encoding (BPE) [46] with a 49 152 vocab size. The text sequence is padded with [SOS] and [EOS] tokens. [SOS] and [EOS] is padded at the beginning and end of the text sequence, respectively. The final text feature representation is the activation from the last layer of the transformer that corresponds to the [EOS] token. This text representation is later normalized by layer normalization and linearly projected into the joint video-text embedding space.

3.1.2 Video encoder. Our video encoder is a vision transformer (ViT), which first successfully applied transformers in vision tasks. The architecture of ViT is the same as the transformer in natural language processing, except ViT introduces an additional visual tokenization process to convert images into discrete sequences. When feeding images or videos into a ViT, we first convert the non-overlapped image patches into visual sequences, where a [CLASS] token is prepended to the beginning of sequences as BERT [40]. Then the output of [CLASS] token at the final layer is extracted as the visual representation. In this work, we adopt a 2D linear projection to project image patches of different frames into an embedding space independently following the practice of CLIP4clip [32]. For convenience, we name this linear transformation process as *visual tokenization*. Generally, we use a ViT-B/32 model [11] with 12 layers and 512 width with 8 attention heads. ViT-B/32 means the non-overlapped input image patch size is 32×32 .

When applying the visual tokenization process to videos, it inevitably produces many redundant tokens as shown in Figure 1. Generally, an input video v_i consists of many temporal related frames: $v_i = \{v_i^1, v_i^2, \dots, v_i^{|v_i|}\}$, where $|v_i|$ is the number of frames in v_i . After visual tokenization, if each frame produces L tokens, the number of visual tokens is $L|v_i|$ (do not consider [CLASS] token). It shows that the number of visual tokens is linear to the number of tokens per frame (L) and the video length. Given an input frame with a size of 224×224 , $L = 49$ for the ViT-B/32 model and $L = 196$ for the ViT-B/16 model. With a larger L , the number of visual tokens becomes much larger. When performing text-video retrieval on long videos, the total number of tokens for a video is large. For example, videos in the ActivityNet [12] dataset usually have a few minutes duration. In this case, $L|v_i|$ will be easily larger than 1 000.

The redundant tokens considerably increase computation costs. To make the training and inference of text-video retrieval models more efficient, we propose to use clustering algorithms to find the most representative token embeddings. This process significantly reduces the number of tokens while maintaining the most valuable information of original tokens. After clustering, we only reserve the center tokens and remove other non-center tokens. The reserved tokens contain most of the information about the video and it is sufficient for text-video retrieval. We describe our multi-segment token clustering in the next section.

3.2 Multi-segment Token Clustering

The overall framework of our video encoder can be found in Figure 2. We perform a multi-segment clustering strategy on visual tokens from a certain temporal segment. This is based on the assumption that neighbor frames are more likely to be the same; then tokens of these similar neighbor frames are more possible to be redundant. Our multi-segment token clustering method empowers the model to achieve segment-level semantic alignment of text and video representations. Previously, CLIP and CLIP4clip adopt the average of frame features or the late fusion of frame features as the video representation. However, the former loses the temporal information, and the latter is a post-processing step and loses the detail temporal variations at the early stage of the transformer. By clustering across multiple frames within a segment at an early or middle stage, image

patches from different temporal positions can interact with each other via the self-attention mechanism.

Specifically, a video sequence $\{v_i^1, v_i^2, \dots, v_i^{|v_i|}\}$ is divided into S segments $\{s_i^1, s_i^2, \dots, s_i^S\}$. Each segment contains $\frac{|v_i|}{S}$ frames and $\frac{L|v_i|}{S}$ tokens. Then we perform token clustering on these $\frac{L|v_i|}{S}$ tokens segment-wise, namely, clustering for each segment independently. Then the centers of all clusters from one segment, *i.e.*, center tokens, are selected and other non-center tokens are simply dropped. These center tokens are concatenated and arranged according to their original relative spatial-temporal position. Center tokens from the upper-left position and early frames are at the beginning of the new token sequence. Center tokens from the bottom-right position and late frames are at the rear-end of the new token sequence.

Multi-segment token clustering algorithm makes our vision model achieve segment-level temporal modeling and be able to capture the detailed temporal variation of video frames. This allows our methods to achieve segment-level alignment of the text t_i and the video v_i consisted of segments $\{s_i^1, s_i^2, \dots, s_i^S\}$:

$$f(v_i, t_i) = \frac{1}{S} \sum_{j=1}^S h(s_i^j)^T g(t_i). \quad (5)$$

The multi-segment token clustering method has at least two advantages: (1) reducing computation costs by cutting down the number of tokens; (2) achieving segment-level semantic alignment of text and video representations via attention among tokens from different frames within the same temporal segment. As shown in Figure 2, assuming we perform token clustering right after the B -th transformer block and the number of clusters is K (ignore [CLASS] token after pooling), this means the length of the input sequence length of the following $(12 - B)$ transformer blocks become K . Generally, $\frac{L|v_i|}{S} \gg K$, obviously, computational costs are largely reduced. It is worth noting that the clustering module can be inserted at any place of ViT and the clustering procedure can be performed for any times. Clustering at an early stage reduces more computation costs. Next, we discuss two clustering methods used in the multi-segment token clustering algorithm.

3.2.1 k-medoids++. In this section, we introduce the first instance of the token clustering method: k-medoids++, a variety of commonly known k-means algorithm. Given a set of tokens $\{x_1, \dots, x_m\} \in \mathbb{R}^d$, where d is the transformer width and $m = \frac{L|v_i|}{S}$ is the number of tokens in a temporal segment, the goal of k-means is to partition the m observations into K ($\leq m$) sets, *i.e.*, $C = \{C_1, C_2, \dots, C_K\}$, so as to minimize the within-cluster sum of squares: $\arg \min_C \sum_{i=1}^K \sum_{x \in C_i} \|x - \mu_i\|_2^2$, where μ_i is the mean of points in C_i , *i.e.*, centroid. Normal k-means contains 4 steps:

1. Initialize cluster centroids $\mu_1, \mu_2, \dots, \mu_K \in \mathbb{R}^d$ randomly;
2. For every i , set $p_i := \arg \min_j \|x_i - \mu_j\|_2^2$;
3. For every j , set $\mu_j := \frac{\sum_{i=1}^m \mathbf{1}_{\{p_i=j\}} x_i}{\sum_{i=1}^m \mathbf{1}_{\{p_i=j\}}}$; $\mathbf{1}\{\cdot\}$ equals to 1 if and only if the inner condition is true;
4. Repeat step 2 and 3 until convergence.

One disadvantage of the normal k-means is that clustering results are sensitive to the centroids initialization. Bad initialization may lead to the collapse of the clustering. Random initialization is

Algorithm 1: KKZ initialization for k-means [23]

Input: tokens $\{x_1, \dots, x_m\} \in \mathbb{R}^d$, cluster number K
Output: centroids $\{\mu_1, \mu_2, \dots, \mu_K\} \in \mathbb{R}^d$

```

1  $\mu_1 \leftarrow \arg \max_{x_i} \|x_i\|_2;$ 
2 for  $i \leftarrow 2$  to  $K$  do
3   for  $j \leftarrow 1$  to  $m$  do
4     for  $k \leftarrow 1$  to  $i$  do
5       // calculate distance between  $x_j$  and  $\mu_k$ 
6        $d_{j,k} \leftarrow \text{distance}(x_j, \mu_k);$ 
7     end
8      $d_j \leftarrow \min_k d_{j,k};$ 
9   end
10   $\mu_i \leftarrow \arg \max_{x_j} d_j;$ 
11 end

```

also not suitable for retrieval, as we would obtain inconsistent retrieval results when querying multiple times. Therefore, we need a deterministic centroids initialization method. In this work, we adopt the KKZ initialization method [23, 48]. The algorithm is shown in Algorithm 1. It first chooses the point with the maximum ℓ_2 -norm as the first centroid, then chooses the point with the maximum distance to the existing centroids as the next centroids. The algorithm is simple but effective [23, 48]. It makes k-means deterministic and accelerates its convergence speed.

In our token clustering process, we use medoids rather than centroids. Namely, we choose the nearest point to centroids as the seed point in steps 2&3 in the normal k-means methods. This is because the semantic of mean pooling representation of tokens within a cluster may shift away from the exact token embeddings. Combining k-medoids and KKZ initialization, we get k-medoids++ – the name philosophy follows k-mean++ [1]. The complexity of k-medoids++ is $O(mKdI)$, where I is the iteration upper bound and $O(d)$ for computing the distance between two points in $\mathbb{R}^d$.

3.2.2 Spectral clustering. K-means is suitable for spherical data clusters. However, our data points are in a high dimension space $\mathbb{R}^d$ ($d = 512$ in most cases), and the data distribution is unknown. The shape of data clusters may not be spherical. To resolve this underlying problem, we further introduce spectral clustering into the token clustering process. Spectral clustering is a graph partition method that aims to maximize the weights of connections within groups and minimize the weights of connections between groups. It first needs to construct the graph $G = (X, E)$, where X is the vertex set and each vertex represents a data point x , E is the edge set and each edge denotes the (weighted) connection between two vertices. In this work, we use the normalized spectral clustering algorithm described in [36]. The algorithm contains 5 steps:

1. Construct similarity graph. Let W be its weighted adjacency matrix, D be the degree matrix;
2. Compute normalized Laplacian $L_{sym} = D^{-\frac{1}{2}}(D - W)D^{-\frac{1}{2}}$;
3. Compute the first K eigenvectors $\mu_1, \dots, \mu_K$ of L_{sym} which correspond to the first K least eigenvalues;
4. Let $U = [\mu_1, \dots, \mu_K]$; Normalize each row of U to have norm of 1, generally, ℓ_2 norm is used;

Algorithm 2: sign flip for SVD [5]

Input: $L \in \mathbb{R}^{m \times m}$, truncated singular value decomposition (U, Σ, V) of L , $U = [u_1, \dots, u_K] \in \mathbb{R}^{m \times K}$
Output: $U' = [u'_1, \dots, u'_K]$ with appropriate signs

```

1 for  $k \leftarrow 1$  to  $K$  do
2    $Y = L - \sum_{i=1, i \neq k}^K \sigma_i u_i u_i^T;$ 
3   /*  $\text{sign}(\cdot)$  return the sign of input,  $Y_{\cdot,j}$ 
   denote the  $j$ -th column of  $Y$  */
4    $s_k = \sum_{j=1}^m \text{sign}(u_k^T Y_{\cdot,j})(u_k^T Y_{\cdot,j})^2;$ 
5 end
6 for  $k \leftarrow 1$  to  $K$  do
7    $u'_k = \text{sign}(s_k)u_k;$ 
8 end

```

5. Consider each row of U as a new data point, apply k-means to these data points.

The above algorithm first performs dimension reduction and then data clustering. In this work, we use SVD to solve the eigenvectors of L_{sym} as $L_{sym} = L_{sym}^T$. A sign correct algorithm is further introduced to resolve the sign ambiguity in SVD as the direction of points after dimension reduction also matters for some distance metrics, e.g., ℓ_2 distance. The main idea of this sign correct algorithm is to make the direction of singular vectors aligns with the majority direction of data points [5], i.e., the sign of the sum of the inner product of singular vectors and data points should be positive. Here we describe the special case of this algorithm for symmetric matrix in Algorithm 2. The complexity of spectral clustering is $O(mK^2I + m^3)$, where $O(m^3)$ for step 1-4 and $O(mK^2I)$ for step 5. For more details about spectral clustering refer to the tutorial [52].

4 EXPERIMENTS

4.1 Experimental Details

Datatest. We validate our model on four datasets: MSR-VTT [60], MSVD [7], LSMDC [42], and ActivityNet [12]. To save computational costs, the shorter side of videos are resized to 224 and the frame per second (fps) is set to 3. **(a) MSR-VTT** contains 10 000 videos with a length ranges from 10 ~ 32 seconds and 200 000 captions. We use two types of data splits, training-7K and training-9K, to compare with baselines. The training-7K follows the data splits from HowTo100M [35] and the training-9K follows the data splits from [13]. The test data in both splits is ‘test 1k-A’, which contains 1 000 video-text pairs following JSFusion [63]. If we do not specify, we use training-9K as the default. **(b) MSVD** contains 1 970 videos with a duration ranges from 1 ~ 62 seconds. Train, validation, and test splits contain 1 200, 100, and 670 videos, respectively. Each video has approximately 40 associated sentences in English. **(c) LSMDC** is comprised of 118 081 videos that ranges from 2~30 seconds. The videos were extracted from 202 movies. The validation set contains 7 408 videos. The 1 000 videos in the test set are from movies independent from the training and validation splits. **(d) ActivityNet** [12, 17] consists of 20 000 YouTube videos, and some of them are minutes long. We follow [13, 67] to concatenate all the

Method	MeM. GB	Speed ms	Text → Video					Video → Text				
			R@1↑	R@5↑	R@10↑	MdR↓	MnR↓	R@1↑	R@5↑	R@10↑	MdR↓	MnR↓
CE [29]	-	-	19.8	49.0	63.8	6	-	-	-	-	-	-
TT-CE+ [10]	-	-	25.4	56.9	71.3	4	-	27.1	55.3	67.1	4	-
Frozen in Time [2]	-	-	33.7	64.7	76.3	3	-	-	-	-	-	-
CLIP zero-shot	-	-	37.0	64.1	73.8	3	-	59.9	85.2	90.7	1	-
CLIP4clip (meanP) [32]	20.8	24.4	46.2	76.1	84.6	2	10.0	56.6	79.7	84.3	1	7.6
CLIP4clip (seqTransf)	-	-	45.2	75.5	84.3	2	10.3	62.0	87.3	92.6	1	4.3
baseline (CLIP4clip (meanP), ViT-B/32)	20.8	24.4	45.9	74.9	84.7	2	10.4	51.0	76.3	82.2	1	9.1
CenterCLIP (k-medoids++, $B_6 - 4, 49$)	15.0	22.9	47.6	77.5	86.0	2	9.8	54.2	78.4	84.9	1	7.6
CenterCLIP (k-medoids++, $B_6 - 3, 49$)	14.2	22.9	47.3	76.8	85.6	2	9.9	57.9	83.6	90.5	1	5.2
CenterCLIP (spectral, $B_6 - 4, 49$)	14.9	40.8	47.4	76.5	85.2	2	9.7	62.7	88.1	92.8	1	4.1
CenterCLIP (spectral, $B_6 - 3, 49$)	14.2	43.6	47.3	76.9	86.0	2	9.7	63.5	86.4	92.6	1	3.8
baseline (CLIP4clip (meanP), ViT-B/16)	25.7	59.6	49.6	79.5	88.0	2	8.6	62.7	83.9	89.4	1	6.1
CenterCLIP (k-medoids++, $B_6 - 4, 160$)	17.6	86.5	50.6	80.3	88.4	1	8.4	68.4	90.1	95.0	1	3.0

Table 1: Results on MSVD. MeM. is the average GPU memory cost when training on 2 and 8 Tesla V100 GPUs for ViT-B/32 and ViT-B/16, respectively. Speed is the inference time per video during evaluation on a Tesla V100 GPU.

Method	MeM. GB	Speed ms	Text → Video					Video → Text				
			R@1↑	R@5↑	R@10↑	MdR↓	MnR↓	R@1↑	R@5↑	R@10↑	MdR↓	MnR↓
FSE [67]	-	-	18.2	44.8	-	7.0	-	16.7	43.1	-	7.0	-
CE [29]	-	-	18.2	47.7	-	6.0	12.1	17.7	46.6	-	6.0	24.4
CMGSD [15]	-	-	24.2	56.3	-	4.0	-	24.6	56.8	-	4.0	-
MMT [13]	-	-	28.7	61.4	-	3.3	16.0	28.9	61.1	-	4.0	17.1
TT-CE+ [10]	-	-	23.5	57.2	-	4.0	-	23.0	56.1	-	4.0	-
T2VLAD [56]	-	-	23.7	55.5	-	4.0	-	24.1	56.6	-	4.0	-
SSB [38]	-	-	29.2	61.6	-	3.0	-	28.7	60.8	-	2.0	-
CLIP zero-shot	-	-	21.7	46.0	59.6	7.0	39.7	17.9	40.8	54.2	8.0	43.3
CLIP4clip (meanP) [32]	25.0	82.0	40.5	72.4	-	2.0	7.4	42.5	74.1	85.8	2.0	6.6
CLIP4clip (seqTransf)	-	-	40.5	72.4	-	2.0	7.5	41.4	73.7	85.3	2.0	6.7
baseline (CLIP4clip (meanP), ViT-B/32)	25.0	82.0	41.8	73.9	84.7	2.0	7.3	42.8	73.8	85.3	2.0	6.9
CenterCLIP (k-medoids++, $B_6 - 15, 49$)	16.8	71.3	43.9	75.3	85.2	2.0	7.0	44.2	75.0	86.1	2.0	6.8
CenterCLIP (k-medoids++, $B_6 - 12, 49$)	16.2	70.4	43.5	75.0	85.9	2.0	6.9	44.5	75.3	86.0	2.0	6.7
CenterCLIP (spectral, $B_6 - 20, 49$)	17.7	162	43.5	75.1	85.4	2.0	6.9	44.1	75.1	86.0	2.0	6.7
CenterCLIP (spectral, $B_6 - 15, 49$)	16.8	174	43.9	74.6	85.8	2.0	6.7	44.5	75.7	86.2	2.0	6.5
CenterCLIP (k-medoids++, $B_6 - 15, 160$, ViT-B/16)	23.0	419	46.2	77.0	87.6	2.0	5.7	46.7	77.1	88.0	2.0	5.5

Table 2: Results on ActivityNet. MeM. is the average GPU memory cost when training on 8 and 32 Tesla V100 GPUs. Baseline with ViT-B/16 OOM on 32 Tesla V100 GPUs. Speed is the inference time per video during evaluation on a Tesla V100 GPU.

descriptions of a video to form a paragraph and evaluate the model with video-paragraph retrieval on the val1 split.

Learning strategies. We apply warm up and cosine learning rate decay policy [14, 16]. If the initial learning rate is lr and current epoch is $epoch$, for the first $slow_epoch$ steps, the learning rate is $lr \times \frac{epoch}{slow_epoch}$; for the rest epochs, the learning rate is $0.5 \times lr \times (1 + \cos(\pi \times \frac{epoch - slow_epoch}{max_epoch - slow_epoch}))$. Generally, lr is $1e-5$ for ActivityNet and $5e-6$ for other datasets; max_epoch is 8 for ActivityNet and 5 for other datasets; $slow_epoch = 0.1 \times max_epoch$. AdamW [31] optimizer is adopted with decoupled weight decay value 0.2.

Sequence length and batch size. For ActivityNet, the maximal text sequence length is 77 and the frame length is 60. For other datasets, the maximal text sequence length is 32 and the frame length is 12. The total batch size is always 128. Experiments with ViT-B/32 for ActivityNet are done on 8 NVIDIA Tesla V100 GPUs. Experiments with ViT-B/32 for other datasets need at least 2 RTX 3090 GPUs. All experiments are done with mixed precision [33].

Frame sampling. We adopt a sparse sampling strategy following TSN [54]. During training, video frames are divided into N_{in} segments, and we randomly sample one frame from each segment. During the evaluation, N_{in} frames are uniformly sampled from the video. As the above said, $N_{in} = 60$ for ActivityNet and $N_{in} = 12$ for the other datasets. These N_{in} frames are further divided into S segments during token clustering.

Evaluation metric. We use standard retrieval metrics: recall at rank $\mathbb{K}$ ($R@K$, higher is better), median rank (MdR, lower is better), and mean rank (MnR, lower is better) to evaluate the performance. $R@K$ (Recall at K) calculates the percentage of test samples for which the correct result is found in the top- K retrieved points to the query sample. $R@1$, $R@5$, and $R@10$ are reported. Median Rank calculates the median of ground-truth results in the ranking. Mean Rank calculates the average rank of all correct results.

Setting of CenterCLIP. We use $(B_a - S, K)$ to represent the setting. It means we perform token clustering right after the a -th transformer block, the number of temporal segments is S , and the

Method	MeM. GB	Speed ms	Text → Video					Video → Text				
			R@1↑	R@5↑	R@10↑	MdR↓	MnR↓	R@1↑	R@5↑	R@10↑	MdR↓	MnR↓
CLIP4clip (meanP) [32]	20.8	24.4	42.1	71.9	81.4	2	15.7	-	-	-	-	-
CLIP4clip (seqTransf)	-	-	42.0	68.6	78.7	2	16.2	-	-	-	-	-
baseline (CLIP4clip (meanP), ViT-B/32)	20.8	24.4	42.4	69.2	79.4	2	17.3	42.2	68.8	78.8	2	12.5
CenterCLIP (k-medoids++, $B_6 - 4, 49$)	15.0	22.9	43.7	71.3	80.8	2	16.9	41.8	68.9	77.9	2	13.3
CenterCLIP (k-medoids++, $B_6 - 3, 49$)	14.2	22.9	43.5	68.5	79.7	2	17.7	40.9	68.4	78.3	2	13.4
CenterCLIP (spectral, $B_6 - 4, 49$)	14.9	40.8	43.4	70.5	79.8	2	15.7	42.1	70.5	80.6	2	11.7
CenterCLIP (spectral, $B_6 - 3, 49$)	14.2	43.6	43.7	71.3	80.2	2	16.2	43.2	71.0	80.4	2	12.3
baseline (CLIP4clip (meanP), ViT-B/16)	25.7	59.6	44.7	71.8	81.6	2	14.3	46.5	73.4	82.3	2	11.0
CenterCLIP (k-medoids++, $B_6 - 4, 160$)	17.6	86.5	47.5	74.4	82.5	2	13.7	46.9	73.4	83.2	2	9.3

(a) Training on training-7K

Method	MeM. GB	Speed ms	Text → Video					Video → Text				
			R@1↑	R@5↑	R@10↑	MdR↓	MnR↓	R@1↑	R@5↑	R@10↑	MdR↓	MnR↓
ActBERT [69]	-	-	8.6	23.4	33.1	36	-	-	-	-	-	-
JSFusion [63]	-	-	10.2	31.2	43.2	13	-	-	-	-	-	-
HowTo100M [35]	-	-	14.9	40.2	52.8	9	-	-	-	-	-	-
CE [29]	-	-	20.9	48.8	62.4	6	-	20.6	50.3	64.0	5.3	-
MMT [13]	-	-	26.6	57.1	69.6	4	24.0	27.0	57.5	69.7	3.7	21.3
T2VLAD [56]	-	-	29.5	59.0	70.1	4	-	31.8	60.0	71.1	3	-
AVLnet [43]	-	-	27.1	55.6	66.6	4	-	28.5	58.6	71.6	3	-
TT-CE+ [10]	-	-	29.6	61.6	74.2	3	-	32.1	62.7	75.0	3	-
CLIP zero-shot	-	-	31.2	53.7	64.2	4	-	27.2	51.7	62.6	5	-
CLIP4clip (meanP) [32]	20.8	24.4	43.1	70.4	80.8	2	16.2	43.1	70.5	81.2	2	12.4
CLIP4clip (seqTransf)	-	-	44.5	71.4	81.6	2	15.3	42.7	70.9	80.6	2	11.6
baseline (CLIP4clip (meanP), ViT-B/32)	20.8	24.4	43.0	70.7	80.6	2	16.2	43.1	70.8	80.6	2	11.4
CenterCLIP (k-medoids++, $B_6 - 4, 49$)	15.0	22.9	43.6	71.4	81.2	2	15.3	42.9	70.4	80.8	2	10.8
CenterCLIP (k-medoids++, $B_6 - 3, 49$)	14.2	22.9	44.0	70.7	81.4	2	15.7	42.9	71.4	81.7	2	11.1
CenterCLIP (spectral, $B_6 - 4, 49$)	14.9	40.8	43.6	71.7	80.6	2	15.4	43.5	72.1	82.2	2	11.1
CenterCLIP (spectral, $B_6 - 3, 49$)	14.2	43.6	44.2	71.6	82.1	2	15.1	42.8	71.7	82.2	2	10.9
baseline (CLIP4clip (meanP), ViT-B/16)	25.7	59.6	45.6	71.2	80.9	2	15.2	43.2	72.5	80.7	2	10.9
CenterCLIP (k-medoids++, $B_6 - 4, 160$)	17.6	86.5	48.4	73.8	82.0	2	13.8	47.7	75.0	83.3	2	10.2

(b) Training on training-9K

Table 3: Results on MSR-VTT. MeM. is the average GPU memory cost when training on 2 and 8 Tesla V100 GPUs for ViT-B/32 and ViT-B/16, respectively. Speed is the inference time per video during evaluation on a Tesla V100 GPU.

Method	MeM. GB	speed ms	Text → Video					Video → Text				
			R@1↑	R@5↑	R@10↑	MdR↓	MnR↓	R@1↑	R@5↑	R@10↑	MdR↓	MnR↓
JSFusion [63]	-	-	9.1	21.2	34.1	36.0	-	12.3	28.6	38.9	20.0	-
CE [29]	-	-	11.2	26.9	34.8	25.3	96.8	-	-	-	-	-
MMT [13]	-	-	12.9	29.9	40.1	19.3	75.0	12.3	28.6	38.9	20.0	76.0
Frozen in Time [2]	-	-	15.0	30.8	39.8	20.0	-	-	-	-	-	-
TT-CE+ [10]	-	-	17.2	36.5	46.3	13.7	-	17.5	36.0	45.0	14.3	-
CLIP zero-shot	-	-	15.1	28.3	35.8	31.0	132	7.5	18.4	25.1	58.0	151
CLIP4clip (meanP) [32]	20.8	24.4	20.7	38.9	47.2	13.0	65.3	20.6	39.4	47.5	13.0	56.7
CLIP4clip (seqTransf)	-	-	22.6	41.0	49.1	11.0	61.0	20.8	39.0	48.6	12.0	54.2
baseline (CLIP4clip (meanP), ViT-B/32)	20.8	24.4	20.1	40.2	48.4	12.0	57.1	21.2	39.3	48.4	12.0	50.8
CenterCLIP (k-medoids++, $B_6 - 6, 49$)	16.4	23.9	21.9	41.1	50.7	10.0	55.6	21.1	41.2	50.2	10.0	48.7
CenterCLIP (k-medoids++, $B_6 - 4, 49$)	15.0	22.9	21.7	39.8	49.8	11.0	54.8	21.4	40.3	50.8	10.0	48.4
CenterCLIP (spectral, $B_6 - 6, 49$)	16.4	40.8	21.6	40.9	49.3	11.0	57.2	20.6	39.5	48.8	12.0	51.4
CenterCLIP (spectral, $B_6 - 4, 49$)	15.0	43.6	21.4	39.7	49.4	11.0	55.9	19.5	39.9	48.0	12.0	50.1
baseline (CLIP4clip (meanP), ViT-B/16)	25.7	59.6	24.1	45.0	55.1	8	51.1	22.5	42.9	53.5	9	45.1
CenterCLIP (k-medoids++, $B_6 - 4, 160$)	17.6	86.5	24.2	46.2	55.9	8	47.3	24.5	46.4	55.8	7	41.3

Table 4: Results on LSMDC. MeM. is the average GPU memory cost when training on 2 and 8 Tesla V100 GPUs for ViT-B/32 and ViT-B/16, respectively. Speed is the inference time per video during evaluation on a Tesla V100 GPU.

number of clusters/centers are constant K . Generally, we construct KNN graph with Gaussian similarity function between two points when applying spectral clustering: $\exp(-\|x_i - x_j\|^2 / (2\sigma^2))$. The neighbours of one vertex is $5 \times \{\text{the number of frames in a segment}\}$ for ViT-B/32, and plus an additional 5 for ViT-B/16. The variance of the Gaussian function σ is simply set to 2.0. No normalization is applied for token embeddings before performing clustering. Baselines in the experiments use the same setting as CenterCLIP.

4.2 Results on Common Benchmarks

As shown in Table 1, Table 2, Table 3, and Table 4. We achieve SOTA performance on all four datasets. Moreover, we also achieve decent memory usage reduction in all cases and obvious speedup of evaluation in some cases. Specifically, for CenterCLIP (ViT-B/32), we achieve a 32% reduction in memory cost and accelerate the model by 6% of the original speed for MSR-VTT, MSVD, and LSMDC in the best situation. For ActivityNet, the reduction in memory cost is 35% and the speedup of evaluation speed is 14% in the best case. These numbers verify the efficiency of our method. For CenterCLIP (ViT-B/16), as the patch size decreases, the number of tokens increases ($4 \times$ as the number of tokens of ViT-B/32). In this work, the clustering complexity is at least linearly related to the number of data points. Therefore, CenterCLIP does not gain speedup for ViT-B/16. However, CenterCLIP also achieves a 32% reduction in memory cost. In future work, we will introduce faster clustering algorithms to speed up the whole model.

Compared to the baseline, CenterCLIP achieves significant improvement on recall. When using ViT-B/32, for MSVD, the maximal gain of text→video R@1 is 1.7%; for MSR-VTT (training-9K), the number is 1.2%; for LSMDC, it is 1.8%; for ActivityNet, it achieves 2.1% improvement of text→video R@1. When using ViT-B/16, for MSVD, the numbers are 1.0%, 2.8%, and 0.1% for text→video R@1, 5.7%, 5.2%, and 2.0% for video→text R@1. CenterCLIP gains more improvement of video→text retrieval performance in this case. All these results demonstrate the effectiveness of our clustering strategy. It aligns segment semantics of text and video.

It is worth noting that spectral clustering and k-medoids++ achieve similar performance in most cases. This is somehow counter-intuitive as spectral clustering should be more suitable for clustering high-dimensional points. This is possible because the data shape of clusters of token embeddings in high dimensional space is nearly spherical. Spectral clustering does achieve better performance in terms of some metrics, *e.g.*, better R@5 and R@10 on MSR-VTT and ActivityNet, and produces the best video→text results on MSVD.

4.3 Diagnostic Experiments

In this section, we will analyze CenterCLIP thoroughly. All diagnostic experiments are taken with CenterCLIP (ViT-B/32).

4.3.1 More baselines. We provide four more strong baselines: 1) pooling of nearby tokens in a temporal segment, after pooling, we get K average tokens for one segment; 2) sparse sampling of tokens in a temporal segment, namely, randomly sample K tokens from a temporal segment during training and uniformly sample K tokens during validation; 3) temporal shift described in TSM [54], here we apply temporal shift to the tokens except [CLASS] embedding; 4) token shift described in [68], the method only shift the [CLASS]

Method	Mem.	T↔V	R@1↑	R@5↑	R@10↑	MdR↓	MnR↓
MSR-VTT (train on training-7K)							
pooling ($B_6 - 6, 49$)	16.39	T→V	41.9	66.6	76.7	2	18.7
		V→T	40.2	65.6	75.8	2	14.4
pooling ($B_6 - 4, 49$)	14.95	T→V	40.6	65.6	75.8	2	17.5
		V→T	40.6	67.3	77.3	2	14.6
sparse sampling ($B_6 - 6, 49$)	16.39	T→V	42.6	68.4	78.4	2	17.6
		V→T	41.6	68.3	77.5	2	12.8
sparse sampling ($B_6 - 4, 49$)	14.95	T→V	42.3	69.1	78.6	2	17.6
		V→T	40.3	66.7	77.0	2	13.5
token shift [68]	20.77	T→V	42.5	68.5	79.6	2	16.4
		V→T	43.3	70.1	80.8	2	12.2
temporal shift [54]	20.77	T→V	34.2	61.8	73.7	3	21.9
		V→T	31.5	61.8	72.2	3	18.2
CenterCLIP ($B_6 - 6, 49$)	16.39	T→V	43.3	69.9	78.6	2	17.7
		V→T	41.8	68.9	77.3	2	12.7
CenterCLIP ($B_6 - 4, 49$)	14.95	T→V	43.7	71.3	80.8	2	16.9
		V→T	41.8	70.2	79.8	2	11.8
LSMDC							
sparse sampling ($B_6 - 6, 49$)	16.39	T→V	20.6	38.7	48.6	12.0	59.8
		V→T	20.4	37.9	45.6	13.5	54.2
token shift [68]	20.77	T→V	21.4	42.3	50.2	10.0	55.6
		V→T	21.7	41.6	50.1	10.0	49.6
CenterCLIP ($B_6 - 4, 49$)	14.95	T→V	21.7	39.8	49.8	11.0	54.8
		V→T	21.4	40.3	50.8	10.0	48.4
ActivityNet							
token shift [68]	24.98	T→V	42.0	73.6	84.8	2.0	7.3
		V→T	42.5	74.3	85.2	2.0	7.0
CenterCLIP ($B_6 - 15, 49$)	16.75	T→V	43.9	75.3	85.2	2.0	7.0
		V→T	44.2	75.0	86.1	2.0	6.8

Table 5: Comparison with strong token selection baselines.

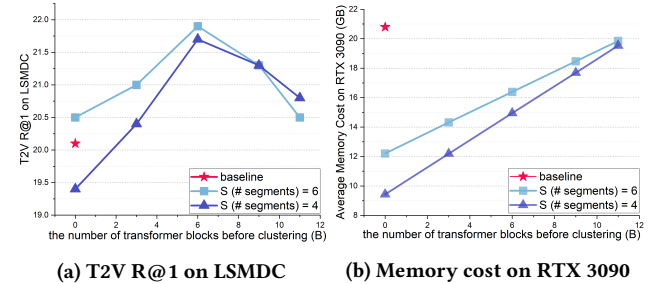


Figure 3: Influence of places of token clustering.

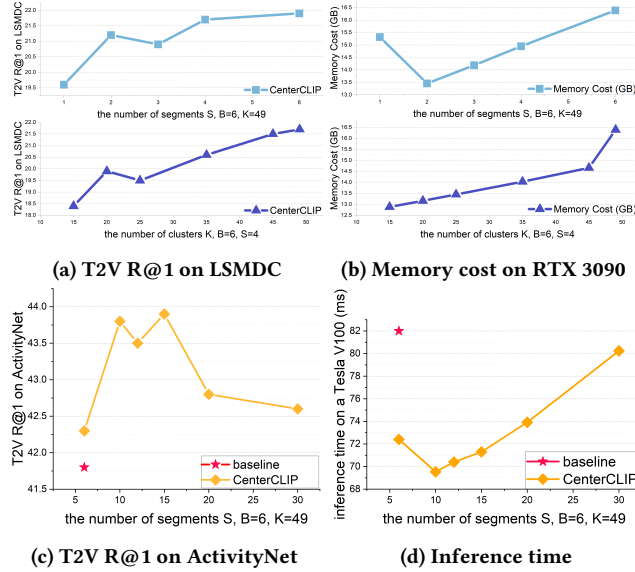
embedding. The shift is performed twice in each transformer block, right before MHSA and FFN. Results are shown in Table 5. Shifting all image patches does not work here. sparse sampling produces a little better results than baseline on MSR-VTT and LSMDC. However, CenterCLIP is much better than sparse sampling, this demonstrates the necessity of selecting representative tokens. Tokenshift achieves pretty good performance on short videos, nevertheless, it does not reduce any computational costs.

4.3.2 The place of performing token clustering. The influence of places of token clustering (k-medoids++) is shown in Figure 3. The smaller the B , the lower the memory cost. The performance of the whole model will also decrease along with the decreasing or increasing of B . A good trade-off between memory cost and performance achieves at $B = 6$, this is also our default setting.

We can also take multiple times of clustering. For instance, firstly perform clustering with ($S = 6, B = 4$) and then with ($S = 3, B =$

Method	Mem.	T $\leftrightarrow$ V	R@1 $\uparrow$	R@5 $\uparrow$	R@10 $\uparrow$	MdR $\downarrow$	MnR $\downarrow$
CenterCLIP	13.52	T $\rightarrow$ V	21.0	42.7	51.9	9.0	57.0
($B_4 - 6, 49$), ($B_8 - 3, 49$)		V $\rightarrow$ T	20.9	41.8	51.5	9.0	50.7
CenterCLIP ($B_6 - 4, 49$)	16.39	T $\rightarrow$ V	21.7	39.8	49.8	11.0	54.8
		V $\rightarrow$ T	21.4	40.3	50.8	10.0	48.4
CenterCLIP ($B_6 - 6, 49$)	14.95	T $\rightarrow$ V	21.9	41.1	50.7	10.0	55.6
		V $\rightarrow$ T	21.1	41.2	50.2	10.0	48.7

Table 6: Performing clustering twice on LSMDC.

Figure 4: Influence of cluster number K and segment S .

8). The results are shown in Table 6. Such progressive clustering strategy achieves pretty good R@5, R@10, MdR, and memory cost reduction. However, performing multiple times will increase the time complexity and this is not suitable for large amounts of tokens. Thus we generally perform clustering once in this work.

4.3.3 The number of cluster K and segment S . We perform experiments on LSMDC and ActivityNet. The results including R@1, memory cost, and inference time are shown in Figure 4. Along with the increase of K , the performance increases, and computation costs also increase. At the same time, a small segment number S does not always achieve better performance, e.g., $S = 1$ on LSMDC and $S = 6$ on ActivityNet. A small segment number means more tokens are dropped. This will cause the loss of more information. When $S = 1$ on LSMDC and $S = 6$ on ActivityNet, the number of tokens in a segment is large, i.e., 12×49 and 10×49 , this leads to more computational costs of clustering as shown in Figure 4c and Figure 4d. Thus a moderate segment number S is usually adopted.

4.3.4 The number of input frames N_{in} . We change the number of input video frames and take experiments with CenterCLIP ($B_6 - 15, 49$) on ActivityNet. The results are shown in Table 7. The large the number of input frames N_{in} , the more computation costs, and a small number of frames will lead to worse performance. Similar ablations about the input of frames on short video datasets like

CenterCLIP ($B_6 - 15, 49$)	Mem.	T $\leftrightarrow$ V	R@1 $\uparrow$	R@5 $\uparrow$	R@10 $\uparrow$	MdR $\downarrow$	MnR $\downarrow$
$N_{in} = 75$	19.48	T $\rightarrow$ V	43.8	74.8	85.8	2.0	6.7
		V $\rightarrow$ T	44.2	75.5	86.8	2.0	6.5
$N_{in} = 60$	16.75	T $\rightarrow$ V	43.9	75.3	85.2	2.0	7.0
		V $\rightarrow$ T	44.2	75.0	86.1	2.0	6.8
$N_{in} = 45$	14.03	T $\rightarrow$ V	42.6	74.5	84.8	2.0	7.0
		V $\rightarrow$ T	43.7	74.5	85.4	2.0	6.7
$N_{in} = 30$	11.29	T $\rightarrow$ V	43.2	74.5	85.1	2.0	6.9
		V $\rightarrow$ T	43.4	74.7	85.5	2.0	6.7

Table 7: Influence of input frames N_{in} on ActivityNet.

Method	Mem.	T $\leftrightarrow$ V	R@1 $\uparrow$	R@5 $\uparrow$	R@10 $\uparrow$	MdR $\downarrow$	MnR $\downarrow$
CenterCLIP ($B_6 - 4, 49$)	16.39	T $\rightarrow$ V	21.6	40.5	50.9	10.0	55.8
w. ℓ_2 norm		V $\rightarrow$ T	22.1	40.8	50.5	10.0	48.4
CenterCLIP ($B_6 - 6, 49$)	14.95	T $\rightarrow$ V	21.5	40.8	50.0	10.5	56.7
w. ℓ_2 norm		V $\rightarrow$ T	20.2	39.8	48.6	12.0	50.5
CenterCLIP ($B_6 - 4, 49$)	16.39	T $\rightarrow$ V	21.7	39.8	49.8	11.0	54.8
wo. ℓ_2 norm		V $\rightarrow$ T	21.4	40.3	50.8	10.0	48.4
CenterCLIP ($B_6 - 6, 49$)	14.95	T $\rightarrow$ V	21.9	41.1	50.7	10.0	55.6
wo. ℓ_2 norm		V $\rightarrow$ T	21.1	41.2	50.2	10.0	48.7

Table 8: Performing k-medoids++ clustering with and without ℓ_2 normalization on LSMDC.

lr	epochs	T $\leftrightarrow$ V	R@1 $\uparrow$	R@5 $\uparrow$	R@10 $\uparrow$	MdR $\downarrow$	MnR $\downarrow$
5e-6	5	T $\rightarrow$ V	40.6	72.3	83.7	2.0	7.7
		V $\rightarrow$ T	41.8	73.1	84.5	2.0	7.0
1e-5	5	T $\rightarrow$ V	41.0	73.4	84.2	2.0	7.3
		V $\rightarrow$ T	42.8	73.8	85.2	2.0	6.9
1e-5	8	T $\rightarrow$ V	41.8	73.9	84.7	2.0	7.3
		V $\rightarrow$ T	42.8	73.8	85.3	2.0	6.9

Table 9: Different learning rates and training epochs for CLIP4clip baselines on ActivityNet.

MSR-VTT can be found in CLIP4clip [32]. When the number of segments S is fixed, a large number of input frames N_{in} will also increase computation costs of the clustering process as the number of tokens in one temporal segment increases.

4.3.5 Normalization of token embeddings. People may be curious about the influence of embedding normalization when performing token clustering. We showed results with and without ℓ_2 normalization in Table 8. The difference is trivial. Indeed, different choices of normalization are equal to different choices of distance metrics in clustering. Besides distance metrics, some other factors can also influence the results of clustering, i.e., how to construct graphs in spectral clustering. However, this is not the focus of our work, and we do not make further explorations in this direction.

4.3.6 Learning rate and training epochs. The original CLIP4clip uses a learning rate of $1e-7$. When setting $lr = 1e-7$ on MSR-VTT (training-7K), we get 39.7 T $\rightarrow$ V R@1 with mixed precision [33] and 41.7 T $\rightarrow$ V R@1 without mixed precision on the split ‘test 1k-A’. The corresponding result of CLIP4clip baseline is 42.1 T $\rightarrow$ V R@1. When increasing the learning rate to 5e-6, we get 42.4 T $\rightarrow$ V R@1 with mixed precision on ‘test 1k-A’. As the mixed precision training saves a lot of GPU memory and accelerates the training procedure,

The four-legged creature, half-horse, half-eagle, rises above the trees.



Figure 5: Visualization of centers after token clustering with different number of frames in a temporal segment.

we are stuck with mixed precision and use $lr = 5e-6$ for short video datasets. For ActivityNet, we found a large learning rate with more training epochs brings a better result. This is shown in Table 9. It is possibly because of the large number of different video frames in long videos in ActivityNet and the sparse sampling strategy we used during training. The model needs more training epochs to learn good representations of videos.

4.3.7 Visualization of image patches of center tokens after clustering. We further display visualization results of center tokens after multi-segment clustering with different numbers of video frames within a temporal segment. The results are shown in Figure 5. It is clear that the clustering algorithm reserves the most representative tokens, for example, in the second and third row of Figure 5, tokens of the foreground animal are selected and only part of the tokens of the similar background remains. This verifies our beginning motivation that using a few typical tokens is already enough for learning discriminative features for video representation.

5 CONCLUSION

In this work, we propose a multi-segment clustering algorithm to reduce the number of redundant tokens of continuous video frames, and achieve segment-level alignment of video and text representations for text-video retrieval task. Our method, named CenterCLIP as we only reserve center tokens of token clusters and drop non-center tokens, is based on the knowledge of large-scale image-text pairs pre-trained model – CLIP. We take extensive experiments on four common text-video multi-modal datasets: MSR-VTT, MSVD, LSMDC, and ActivityNet. CenterCLIP achieves state-of-the-art performance on all these four datasets and surpass the old SOTA by a large margin. At the same time, CenterCLIP realizes a decent reduction in memory costs and speedup of inference time.

ACKNOWLEDGMENTS

This work is supported by National Key R&D Program of China under Grant No. 2020AAA0108800. Thanks Naiyuan Liu for his helpful discussions.

REFERENCES

- [1] David Arthur and Sergei Vassilvitskii. 2007. k-means++: the advantages of careful seeding. In *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2007, New Orleans, Louisiana, USA, January 7-9, 2007*, Nikhil Bansal, Kirk Pruhs, and Clifford Stein (Eds.). SIAM, 1027–1035.
- [2] Max Bain, Arsha Nagrani, Gül Varol, and Andrew Zisserman. 2021. Frozen in time: A joint video and image encoder for end-to-end retrieval. *arXiv preprint arXiv:2104.00650* (2021).
- [3] Iz Beltagy, Matthew E Peters, and Arman Cohan. 2020. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150* (2020).
- [4] Rishi Bommasani, Drew A Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, et al. 2021. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258* (2021).
- [5] R. Bro, E. Acar, and Tamara G. Kolda. 2008. Resolving the sign ambiguity in the singular value decomposition. *Journal of Chemometrics* 22, 2 (2008), 135–140. <https://doi.org/10.1002/cem.1122>
- [6] Shuhao Cao. 2021. Choose a Transformer: Fourier or Galerkin. In *Thirty-Fifth Conference on Neural Information Processing Systems (NeurIPS 2021)*. arXiv:arXiv:2105.14995 [cs.CL]
- [7] David Chen and William B Dolan. 2011. Collecting highly parallel data for paraphrase evaluation. In *Proceedings of the 49th annual meeting of the association for computational linguistics: human language technologies*. 190–200.
- [8] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. 2019. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509* (2019).
- [9] Krzysztof Choromanski, Valerii Likhoshesterov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, et al. 2020. Rethinking attention with performers. *arXiv preprint arXiv:2009.14794* (2020).
- [10] Ioana Croitoru, Simion-Vlad Bogolin, Marius Lefordecu, Hailin Jin, Andrew Zisserman, Samuel Albanie, and Yang Liu. 2021. Teachtext: Crossmodal generalized distillation for text-video retrieval. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 11583–11593.
- [11] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xi-aohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. 2021. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*.
- [12] Bernard Ghanem Fabian Caba Heilbron, Victor Escorcia and Juan Carlos Nibbles. 2015. ActivityNet: A Large-Scale Video Benchmark for Human Activity Understanding. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 961–970.
- [13] Valentin Gabeur, Chen Sun, Karteek Alahari, and Cordelia Schmid. 2020. Multi-modal transformer for video retrieval. In *Computer Vision—ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part IV* 16. Springer, 214–229.
- [14] Priya Goyal, Piotr Dollár, Ross B. Girshick, Pieter Noordhuis, Lukasz Wesolowski, Aapo Kyrola, Andrew Tulloch, Yangqing Jia, and Kaiming He. 2017. Accurate, Large Mini-batch SGD: Training ImageNet in 1 Hour. *CoRR abs/1706.02677* (2017). arXiv:1706.02677
- [15] Feng He, Qi Wang, Zhifan Feng, Wenbin Jiang, Yajuan Lü, Yong Zhu, and Xiao Tan. 2021. Improving Video Retrieval by Adaptive Margin. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1359–1368.
- [16] Tong He, Zhi Zhang, Hang Zhang, Zhongyue Zhang, Junyuan Xie, and Mu Li. 2018. Bag of Tricks for Image Classification with Convolutional Neural Networks. *CoRR abs/1812.01187* (2018).
- [17] Fabian Caba Heilbron and Juan Carlos Nibbles. 2014. Collecting and Annotating Human Activities in Web Videos. In *Proceedings of International Conference on Multimedia Retrieval*. ACM, 377.
- [18] Yuqi Huo, Manli Zhang, Guangzhen Liu, Haoyu Lu, Yizhao Gao, Guoxing Yang, Jingyuan Wen, Heng Zhang, Baogui Xu, Weihao Zheng, et al. 2021. WenLan: Bridging vision and language by large-scale multi-modal pre-training. *arXiv preprint arXiv:2103.06561* (2021).
- [19] Andrew Jaegle, Sebastian Borgeaud, Jean-Baptiste Alayrac, Carl Doersch, Catalin Ionescu, David Ding, Skanda Koppula, Daniel Zoran, Andrew Brock, Evan Shelhamer, et al. 2021. Perceiver io: A general architecture for structured inputs & outputs. *arXiv preprint arXiv:2107.14795* (2021).
- [20] Yunseok Jang, Yale Song, Youngjae Yu, Youngjin Kim, and Gunhee Kim. 2017. Tgif-qa: Toward spatio-temporal reasoning in visual question answering. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2758–2766.
- [21] Chao Jia, Yinfei Yang, Ye Xia, Yi-Ting Chen, Zarana Parekh, Hieu Pham, Quoc V Le, Yunhsuan Sung, Zhen Li, and Tom Duerig. 2021. Scaling up visual and vision-language representation learning with noisy text supervision. *arXiv preprint arXiv:2102.05918* (2021).
- [22] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. 2020. Transformers are rnns: Fast autoregressive transformers with linear attention. In *International Conference on Machine Learning*. PMLR, 5156–5165.
- [23] I. Katsavounidis, C.-C. Jay Kuo, and Zhen Zhang. 1994. A new initialization technique for generalized Lloyd iteration. *IEEE Signal Processing Letters* 1, 10 (1994), 144–146.
- [24] Dotan Kaufman, Gil Levi, Tal Hassner, and Lior Wolf. 2017. Temporal tessellation: A unified approach for video analysis. In *Proceedings of the IEEE International Conference on Computer Vision*. 94–104.
- [25] Nikita Kitaev, Lukasz Kaiser, and Anselm Levskaya. 2020. Reformer: The efficient transformer. *arXiv preprint arXiv:2001.04451* (2020).
- [26] Thao Minh Le, Vuong Le, Svetha Venkatesh, and Truyen Tran. 2020. Hierarchical conditional relation networks for video question answering. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 9972–9981.
- [27] Jie Lei, Linjie Li, Luowei Zhou, Zhe Gan, Tamara L Berg, Mohit Bansal, and Jingjing Liu. 2021. Less is more: Clipbert for video-and-language learning via sparse sampling. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 7331–7341.
- [28] Linjie Li, Yen-Chun Chen, Yu Cheng, Zhe Gan, Licheng Yu, and Jingjing Liu. 2020. Hero: Hierarchical encoder for video+ language omni-representation pre-training. *arXiv preprint arXiv:2005.00200* (2020).
- [29] Yang Liu, Samuel Albanie, Arsha Nagrani, and Andrew Zisserman. 2019. Use what you have: Video retrieval using representations from collaborative experts. *arXiv preprint arXiv:1907.13487* (2019).
- [30] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. 2021. Swin transformer: Hierarchical vision transformer using shifted windows. *arXiv preprint arXiv:2103.14030* (2021).
- [31] Ilya Loshchilov and Frank Hutter. 2018. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations*.
- [32] Huaishao Luo, Lei Ji, Ming Zhong, Yang Chen, Wen Lei, Nan Duan, and Tianrui Li. 2021. CLIP4Clip: An Empirical Study of CLIP for End to End Video Clip Retrieval. *CoRR abs/2104.08860* (2021). arXiv:2104.08860
- [33] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory F. Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. 2018. Mixed Precision Training. In *ICLR*.
- [34] Antoine Miech, Jean-Baptiste Alayrac, Lucas Smaira, Ivan Laptev, Josef Sivic, and Andrew Zisserman. 2020. End-to-end learning of visual representations from uncurated instructional videos. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 9879–9889.
- [35] Antoine Miech, Dimitri Zhukov, Jean-Baptiste Alayrac, Makarand Tapaswi, Ivan Laptev, and Josef Sivic. 2019. Howto100m: Learning a text-video embedding by watching hundred million narrated video clips. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2630–2640.
- [36] Andrew Y. Ng, Michael I. Jordan, and Yair Weiss. 2001. On Spectral Clustering: Analysis and an algorithm. In *ADVANCES IN NEURAL INFORMATION PROCESSING SYSTEMS*. 849–856.
- [37] Or Patashnik, Zongze Wu, Eli Shechtman, Daniel Cohen-Or, and Dani Lischinski. 2021. Styleclip: Text-driven manipulation of stylegan imagery. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2085–2094.
- [38] Mandela Patrick, Po-Yao Huang, Yuki Asano, Florian Metz, Alexander Hauptmann, Joao Henriques, and Andrea Vedaldi. 2020. Support-set bottlenecks for video-text representation learning. *arXiv preprint arXiv:2010.02824* (2020).
- [39] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. 2021. Learning Transferable Visual Models From Natural Language Supervision. *CoRR abs/2103.00020* (2021). arXiv:2103.00020
- [40] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog* 1, 8 (2019), 9.
- [41] Yongming Rao, Wenliang Zhao, Benlin Liu, Jiwen Lu, Jie Zhou, and Cho-Jui Hsieh. 2021. DynamicViT: Efficient Vision Transformers with Dynamic Token Sparsification. *arXiv preprint arXiv:2106.02034* (2021).
- [42] Anna Rohrbach, Marcus Rohrbach, and Bernt Schiele. 2015. The long-short story of movie description. In *German conference on pattern recognition*. Springer, 209–221.
- [43] Andrew Rouditchenko, Angie Boggust, David Harwath, Brian Chen, Dhiraj Joshi, Samuel Thomas, Kartik Audhkhasi, Hilde Kuehne, Rameswar Panda, Rogerio Feris, et al. 2020. Avnet: Learning audio-visual language representations from instructional videos. *arXiv preprint arXiv:2006.09199* (2020).
- [44] Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. 2021. Efficient content-based sparse attention with routing transformers. *Transactions of the Association for Computational Linguistics* 9 (2021), 53–68.
- [45] Michael S Ryoo, AJ Piergiovanni, Anurag Arnab, Mostafa Dehghani, and Anelia Angelova. 2021. TokenLearner: What Can 8 Learned Tokens Do for Images and Videos? *arXiv preprint arXiv:2106.11297* (2021).
- [46] Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. Neural Machine Translation of Rare Words with Subword Units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics, ACL 2016, August 7-12,*

- 2016, Berlin, Germany, Volume 1: Long Papers. The Association for Computer Linguistics.
- [47] Sheng Shen, Liunian Harold Li, Hao Tan, Mohit Bansal, Anna Rohrbach, Kai-Wei Chang, Zhewei Yao, and Kurt Keutzer. 2021. How Much Can CLIP Benefit Vision-and-Language Tasks? *arXiv preprint arXiv:2107.06383* (2021).
 - [48] Ting Su and Jennifer G. Dy. 2007. In search of deterministic methods for initializing K-means and Gaussian mixture clustering. *Intell. Data Anal.* 11, 4 (2007), 319–338.
 - [49] Chen Sun, Austin Myers, Carl Vondrick, Kevin Murphy, and Cordelia Schmid. 2019. Videobert: A joint model for video and language representation learning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 7464–7473.
 - [50] Laurens van der Maaten and Geoffrey Hinton. 2008. Visualizing Data using t-SNE. *Journal of Machine Learning Research* 9, 86 (2008), 2579–2605.
 - [51] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett (Eds.). 5998–6008.
 - [52] Ulrike Von Luxburg. 2007. A tutorial on spectral clustering. *Statistics and computing* 17, 4 (2007), 395–416.
 - [53] Junke Wang, Xitong Yang, Hengduo Li, Zuxuan Wu, and Yu-Gang Jiang. 2021. Efficient Video Transformers with Spatial-Temporal Token Selection. *arXiv preprint arXiv:2111.11591* (2021).
 - [54] Limin Wang, Yuanjun Xiong, Zhe Wang, Yu Qiao, Dahua Lin, Xiaoou Tang, and Luc Van Gool. 2016. Temporal segment networks: Towards good practices for deep action recognition. In *European conference on computer vision*. Springer, 20–36.
 - [55] Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. 2020. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768* (2020).
 - [56] Xiaohan Wang, Linchao Zhu, and Yi Yang. 2021. T2vlad: global-local sequence alignment for text-video retrieval. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 5079–5088.
 - [57] Bichen Wu, Chenfeng Xu, Xiaoliang Dai, Alvin Wan, Peizhao Zhang, Zhicheng Yan, Masayoshi Tomizuka, Joseph Gonzalez, Kurt Keutzer, and Peter Vajda. 2020. Visual transformers: Token-based image representation and processing for computer vision. *arXiv preprint arXiv:2006.03677* (2020).
 - [58] Dejing Xu, Zhou Zhao, Jun Xiao, Fei Wu, Hanwang Zhang, Xiangnan He, and Yueting Zhuang. 2017. Video question answering via gradually refined attention over appearance and motion. In *Proceedings of the 25th ACM international conference on Multimedia*. 1645–1653.
 - [59] Hu Xu, Gargi Ghosh, Po-Yao Huang, Dmytro Okhonko, Armen Aghajanyan, Florian Metz, Luke Zettlemoyer, and Christoph Feichtenhofer. 2021. Videoclip: Contrastive pre-training for zero-shot video-text understanding. *arXiv preprint arXiv:2109.14084* (2021).
 - [60] Jun Xu, Tao Mei, Ting Yao, and Yong Rui. 2016. Msr-vtt: A large video description dataset for bridging video and language. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 5288–5296.
 - [61] Hongxu Yin, Arash Vahdat, Jose Alvarez, Arun Mallya, Jan Kautz, and Pavlo Molchanov. 2021. AdaViT: Adaptive Tokens for Efficient Vision Transformer. *arXiv preprint arXiv:2112.07658* (2021).
 - [62] Haonan Yu, Jiang Wang, Zhiheng Huang, Yi Yang, and Wei Xu. 2016. Video paragraph captioning using hierarchical recurrent neural networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 4584–4593.
 - [63] Youngjae Yu, Jongseok Kim, and Gunhee Kim. 2018. A joint sequence fusion model for video question answering and retrieval. In *Proceedings of the European Conference on Computer Vision (ECCV)*. 471–487.
 - [64] Youngjae Yu, Hyungjin Ko, Jongwook Choi, and Gunhee Kim. 2017. End-to-end concept word detection for video captioning, retrieval, and question answering. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 3165–3173.
 - [65] Lu Yuan, Dongdong Chen, Yi-Ling Chen, Noel Codella, Xiyang Dai, Jianfeng Gao, Houdong Hu, Xuedong Huang, Boxin Li, Chunyuan Li, et al. 2021. Florence: A New Foundation Model for Computer Vision. *arXiv preprint arXiv:2111.11432* (2021).
 - [66] Andrew Zhai and Hao-Yu Wu. 2019. Classification is a Strong Baseline for Deep Metric Learning. In *30th British Machine Vision Conference 2019, BMVC 2019, Cardiff, UK, September 9-12, 2019*. 91.
 - [67] Bowen Zhang, Hexiang Hu, and Fei Sha. 2018. Cross-modal and hierarchical modeling of video and text. In *Proceedings of the European Conference on Computer Vision (ECCV)*. 374–390.
 - [68] Hao Zhang, Yanbin Hao, and Chong-Wah Ngo. 2021. Token shift transformer for video classification. In *Proceedings of the 29th ACM International Conference on Multimedia*. 917–925.
 - [69] Linchao Zhu and Yi Yang. 2020. Actbert: Learning global-local video-text representations. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 8746–8755.